

Underwater Acoustic Direction Finding

Team Members: Colin Gregg, Owen Funk, Eli Juvan

Technical Advisor: Dr. Rosa Zhen

2. Abstract

This project sought to design an underwater acoustic direction finding system that could eventually be integrated onto an Autonomous Underwater Vehicle (AUV). This system consists of two components, a transmitter and receiver. The transmitter system can send acoustic pulses underwater, with configurable pulse widths and frequencies between 25 kHz and 40 kHz. The receiver utilizes four hydrophones arranged in a uniform linear array and an STM32 Microcontroller (MCU), that is able to detect the broadside Direction of Arrival (DOA) of an incoming acoustic pulse using the MULTiple Signal Classification (MUSIC) algorithm.

The transmitter successfully produces signals that can travel more than 100 ft in a swimming pool, and the receiver can correctly calculate the DOA of the incoming signal within 15° of accuracy. The design was tested in an anechoic water tank and a pool in Taylor Gymnasium. The system is capable of performing correctly in both environments, however testing demonstrated that the receiver system is sensitive to the testing configuration. The software on the microcontroller (MCU) mitigates some of this sensitivity. Suggestions on further development are provided for identifying the error sources and increasing the reliability of the direction finding results.

3. Introduction & Motivation

Problem

Underwater environments pose a difficulty for image based navigation techniques, which are commonly used by systems to inform autonomous actions. Acoustic pingers provide a secondary navigational option that can be used to mark points of interest (POIs), providing a simpler alternative to SLAM/sonar based navigation techniques [1]. By being able to detect the DOA of an incoming signal, an autonomous system could navigate to the position marked by the pinger with minimal additional sensor input. When fused into a system with several different sensors, finding the DOA can provide an additional sensor input that could be used in conjunction with the existing sensors to provide a more complete view of the environment.

Motivation

In the Robosub competition, two Benthos ALP-365 pingers are used to mark competition tasks [2]. Being able to locate these pingers allows competing teams to more easily navigate to the tasks in order to complete them, scoring more points overall. The Lehigh Underwater Robotics (LUR) team currently lacks the capabilities to detect the DOA of incoming acoustic signals.

Our system seeks to implement an acoustic pinger that is capable of transmitting pulses that emulates the specifications provided by the Robosub competition organizers, and a direction finding system that can calculate the DOA of this incoming signal. Developing the transmitter to pair with the receiver is important as the Benthos ALP-365 is difficult to source, especially for a smaller scale organization like LUR. By creating a transmitter that can generate an easily modifiable pulse, the LUR team could use this system to better test the functionality of their drone, leading to increased scores at the competition.

For the receiver end, our design implements the Multiple Signal Classification algorithm using four hydrophones. This algorithm was first introduced in 1986 [3], and allows for a system to determine the DOA of an incoming signal based on the phase relationship between the different receiver elements. Requiring four hydrophones allows the design to be more easily mounted to the front of the LUR team's vehicle, providing the system with an accurate DOA estimate that could be used during competition runs.

In order to make integration into the overall drone system easier, the receiver utilizes an STM32 F446RE MCU. By utilizing the onboard 2.4 MSPS Analog to Digital Converter (ADC) module [4], the overall footprint of the design is reduced, which is important for future integration into the 6" diameter tube that houses the primary electronics of the drone.

Importance of Work

This project provides the groundwork required for implementing a DOA receiver onto the LUR team's AUV. Many of the top performing Robosub teams, such as Team Bumblebee from the National University of Singapore, have the ability to detect the DOA of the signals from the pingers on the course and navigate to the tasks. By adding this functionality to LUR drone, the team will be more competitive should the team decide to compete in the Robosub competition again.

Outside of improving the performance of the LUR team's vehicle, the MUSIC algorithm was designed around RF signals in air, and to our knowledge there have not been many public projects that seek to adapt this system onto underwater autonomous vehicles. Being able to detect the DOA of acoustic waves underwater has several real world applications [5], including navigating to and checking the status of commercial fishing nets (that have been marked with an acoustic pinger), navigating to and retrieving

black boxes, and locating certain species of mammals in underwater environments. By developing a receiver system that can be integrated onto existing AUVs, this project could improve autonomous navigation to better perform those tasks.

4. System Overview

Our system consists of a transmitter with selectable frequency that transmits pulses under 500ms in duration at a pulse frequency between 0.5Hz and 2Hz and a receiver module to measure the transmitted pulses and calculate an estimated Direction of Arrival (DOA).

Transmitter

The transmitter system is designed around the specific piezoelectric transducer that our group was provided by our advisor. The overall architecture needs to take a small signal generated by a MCU, amplify the signal, and drive the transducer with it. In order to do this, additional circuitry is needed, including digital isolation to make sure none of the high current and high voltage signals in the latter half of the circuit could reach the sensitive MCU and digital circuitry and cause damage. A matching network circuit was also needed in order to convert our amplified signal into something that could actually drive the transducer.

The figure below shows the block diagram for the transmitter, illustrating our implementation of the circuit.

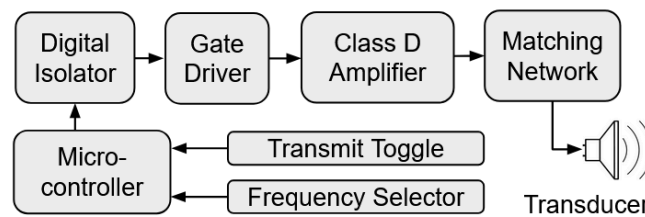


Figure 1: Block Diagram of our System Implementation of the Transmitter Module.

The first design choice was what amplifier to use, and we chose a Class D amplifier. Class D amplifiers are able to generate high power and large voltage signals from a small and easy to generate digital signal due to how they operate. Also, they are extremely efficient, outperforming other power amplifier classes like Class A, Class B, and Class AB.

Figure 2 shows the general theory of operation for the Class D amplifier, specifically for how we implemented it. Figure 2a highlights the wiring of the Gate Driver block from figure 1 to the Class D amplifier, and also expands on the circuit implementation of the Class D Amplifier and Matching Network blocks. The Class D topology used here is a Half-Bridge configuration, defined by the two Power MOSFETs Q1 and Q2. It works by outputting two inverted PWM signals from the Gate Driver, which then turn on either one of the transistors, but never both at once, connecting the output either up to V_{DD} or down to V_{SS} .

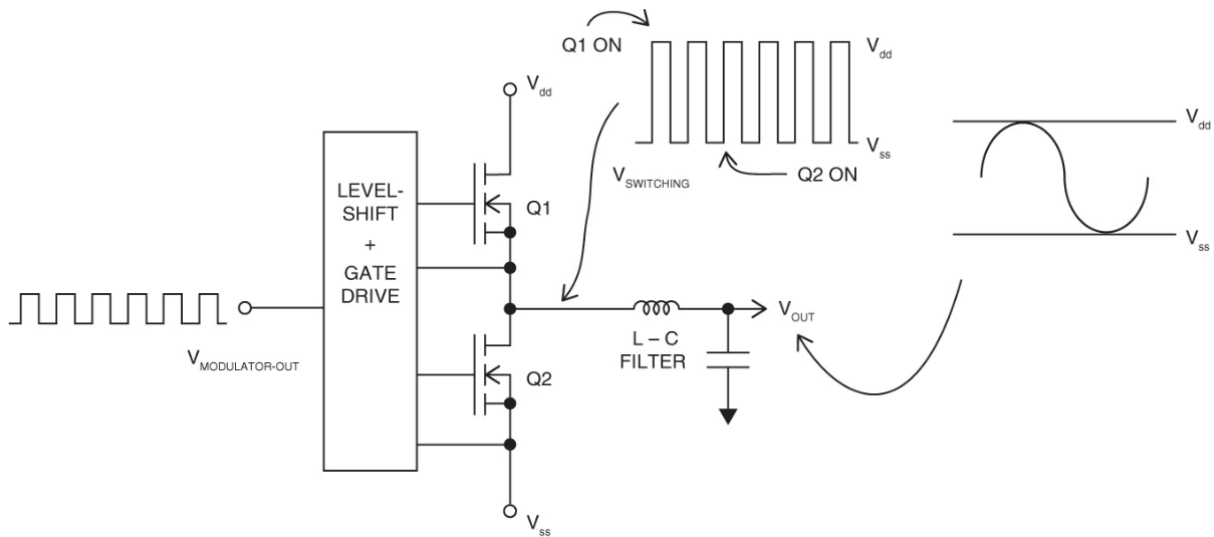


Figure 2a: Typical Class D Amplifier Design.

The matching network we implement is identical to the one shown in figure 2a, where we use a series inductor matched to resonate with the shunt capacitor to ground. However, the capacitor in our demodulation filter is actually the parallel capacitance of the piezoelectric transducer itself, C_0 , as seen in figure 2b. The purpose of the matching network is to create an LC low pass filter which attenuates the higher frequency components of the square wave generated by the input circuit and amplified by the Class D. Setting the corner frequency of this LC filter to just above the desired transmission frequency removes all higher order harmonics turning the square wave into a sine wave, which can then drive the transducer.

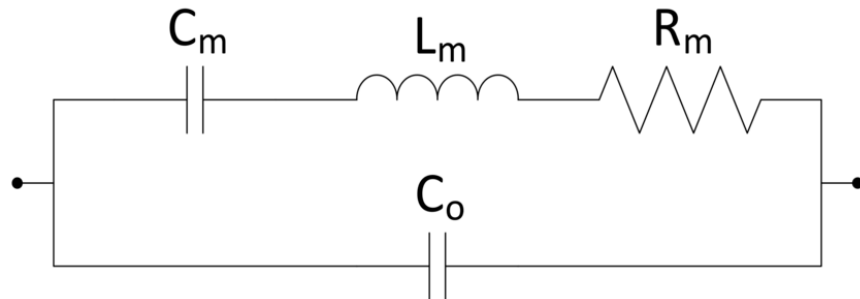


Figure 2b: Butterworth-Van Dyke (BVD) Electrical Model of a Piezoelectric Transducer.

The gate driver circuit has two purposes. The first is to add dead time to the PWM signal, and the second is to quickly charge and discharge the MOSFET for fast and power efficient turn on and off. As seen in figure 2c, dead time ensures that there is no possible overlap between the two inverted PWM signals such that they are both high. If there ever was an edge case where both PWM signals were high, both MOSFETs from figure 2a would turn on and short the upper and lower power rails. Figure 2d highlights the parasitic values of importance to a Power MOSFET, mainly that the gate to source and gate to drain capacitances need to be charged (on turn on) and discharged (on turn off). In order for this to happen quickly, the gate driver circuit needs to be able to both sink and source large amounts of current.

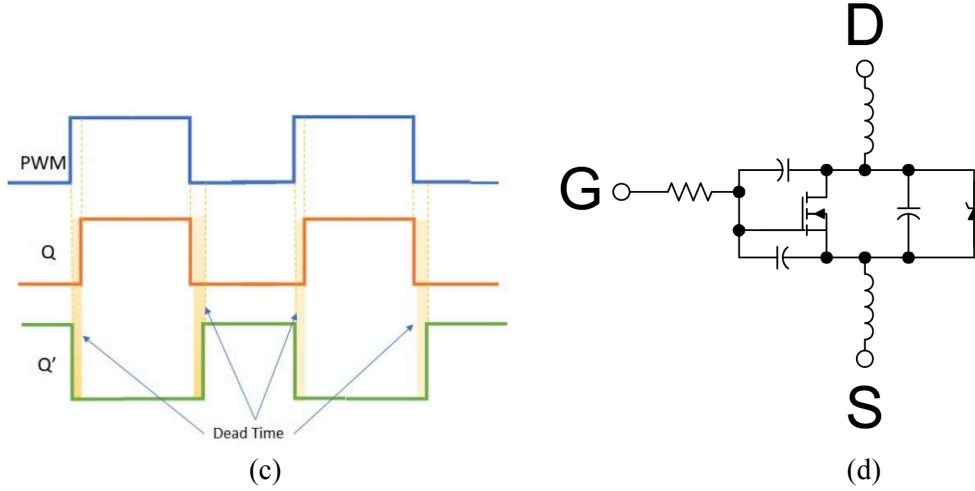


Figure 2c and 2d: (c) deadtime implemented into two inverted PWM signals, which delays the rising edge of one signal so it never overlaps with the falling edge of another signal. [10] (d) Power MOSFET Parasitic Model, showing Gate-to-Source and Gate-to-Drain capacitances [11].

Receiver

Figure 3 shows the stages used to implement our receiver module. The input to the module are four acoustic hydrophones. These receive audio and ultrasonic signals underwater and convert the physical soundwave into electrical signals. Each hydrophone then transmits the electrical signal as a differential twisted-pair to the first stage of the receiver module, which is a Low Noise Amplifier, and each hydrophone has its own associated LNA board.

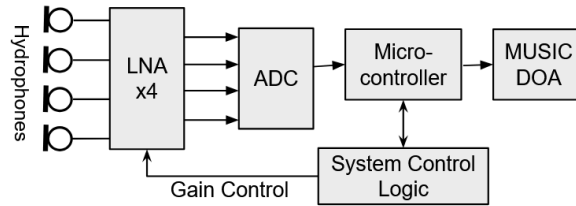


Figure 3: Block Diagram of our System Implementation of the Receiver Module.

The purpose of the LNA is to take the small noisy signal coming from the sensor and amplify it to nearly 1V peak-to-peak, while also passing it through a bandpass filter that removes any signals that are out of the desired bandwidth. After being filtered and amplified, the signal is passed to a variable gain block that can attenuate the signal if it is too large or amplify it again if needed. The variable gain is voltage controlled by an SPI signal from the MCU. The SPI signal is sent to a shift register that loads a Digital-to-Analog converter (DAC) that outputs the correct analog signal to control the variable gain. The last stage of the LNA board is another bandpass filter, and then a level shift circuit that prepares the signal to be sent to the Analog-to-Digital converter (ADC) on the MCU. Once the MCU has sampled the signal, it can perform the MUSIC algorithm to determine an estimated DOA, and it can be told to update the gain control on the LNA boards if needed.

5. Engineering Design & Decisions

Transmitter

The design of our transmitter stemmed from the materials we had on hand, the rules of the Robosub competition, and a principle to make this as simple to use as possible. A major constraint for our project was that we had to use a specific transmitter our advisor had on hand. Underwater acoustic piezoelectric transducers that can transmit a strong enough signal are expensive, much more than our entire budget for the project, so we had to make our driver board work with the one we were given. The first design choice was what topology we should use for the power amplifier. The two main things to consider about our transducer were that it was a piezoelectric device and that it required a very large voltage across it to work. One last thing to consider when choosing our topology for the power amplifier is that this will be a battery powered system in a small, underwater enclosure. With that in mind, the obvious choice to pick was a Class D amplifier configured in a push-pull configuration, and its circuit for this topology can be seen in figure 4. The advantages of this circuit are two fold: the Class-D amplifier is very efficient and does not need large heat sinks making it lightweight, low power, and not overly hot, and the push-pull configuration allows the circuit to step up the voltage through a transformer and configure the load in a Bridge-Tied-Load (BTL) configuration which doubles the voltage and makes the signal more symmetric thereby reducing distortion.

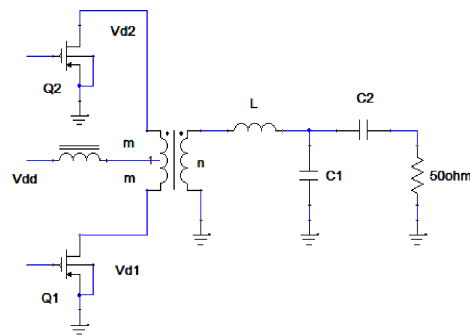


Figure 4: Simplified Class-D Push Pull Circuit [12]

The next major design choice that stemmed from the transmitter we were using was the LC demodulating filter that would connect the output of the power amplifier to the BTL. We call this filter the matching network because it matches the square wave output to our specific transducer and turns the square wave into a sine wave at the correct frequency. To do this, we designed an LC filter that made use of the internal parallel capacitance of the piezoelectric transducer to create a low pass filter. The simplest method, and the one we ended up going with, can be seen in figure 5d, which is series inductors to form an LC low pass filter. Figure 5d shows two inductors, because in a BTL setup there is no common ground, the two output windings of the transformer tie to either end of the load. Several other filter topologies were tested both in simulation and in the lab, and they can be seen in figure 5a, b, and c. These were not selected because they either did not work as well as the LC filter, were unstable, or required too many parts. The inductors for this matching network are large, as regular 0.5W inductors would not perform properly and eventually would burn.

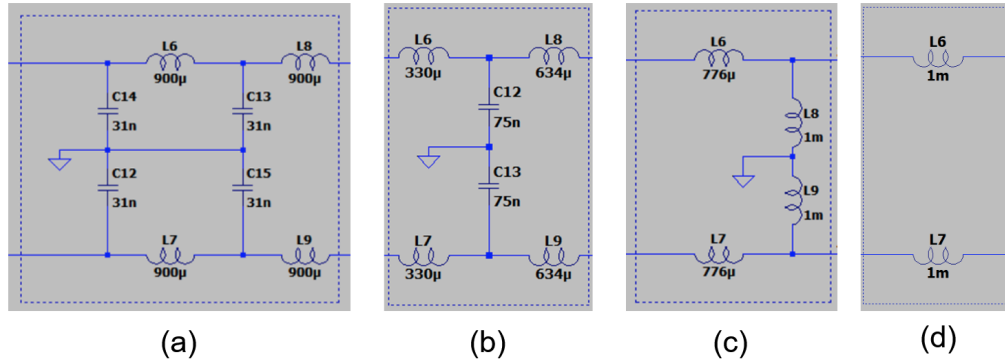


Figure 5: (a) fourth order LC filter with capacitors to shunt common mode distortion to ground. (b) the second order version of the filter from (a). (c) Adding parallel inductors reduced the current drawn into the transducer. (d) series inductors L6 and L7 form a simple LC filter with the parallel capacitance, C_0 , of the transducer load.

After deciding the means to generate a high power sinusoidal signal for our transducer, the last design choice was made to make the use of and implementation of the driver board as easy as possible. The next main choice was the method for signal generation. Typically, Class-D amplifiers use a sawtooth wave and comparator in order to generate a square wave with the right frequency, level shift, and deadtime. However, the circuits to generate this square wave need several IC packages and dozens of passive components, as well as a precision reference voltage. The Si824x Gate Driver chip we used is already able to level shift a signal and program in deadtime to a square wave using just a resistor from one of its pins to ground. That means we only needed to generate a square wave at specific frequencies, and since these frequencies are relatively low, a MCU is more than enough to achieve this. Additionally, the Robosub competition dictates that the transmitted signal is sent out between every 0.5 seconds and 2 seconds, with the duration of each pulse being less than 500ms. The pulsing feature and duration of the PWM signal can also be easily set by a MCU. The MCU we chose was a Tiva TM4C because we had multiple on hand and the entire group already had prior experience with it from a class. Because the tasks needed by the MCU are so simple, it was quick and efficient to use this MCU.

Receiver

The first design choice we made in the project was to use the Multiple Signal Classification (MUSIC) algorithm for the direction of arrival estimation. In the Robosub competition there will be multiple transmitters in the pool, meaning it is necessary for our system to be able to detect the direction of multiple signals. The MUSIC algorithm only needs one additional receiver to detect another signal, making it very efficient and needing less space, less channels, and much lower costs. There are other alternatives like ESPRIT, however, they require many more receivers making it impractical for the LUR team to implement. Additionally, the MUSIC algorithm has been used for 40 years and has many resources to help with implementation on MATLAB, C, and C++.

The LNA boards we received were almost fully designed and implemented, just at a higher frequency. This meant we did not need high-level architectural decisions, but small adjustments were needed. The first step was to redesign the two bandpass filters on the board to now have cutoff frequencies at 20kHz and 45kHz. While designing the filters we made sure that they were Bessel filters in order to ensure the constant group delay for all frequencies. This is important because of the algorithm we decided to

implement, which uses the relative phase difference between the different channels, so any non-linearity in phase delay caused by the filters and amplifiers would have to be accounted for.

Another major design choice was to use four hydrophones as our receivers, configured in a uniform linear array. In order to implement the MUSIC algorithm, we have to create something called the steering vector which describes how our receiver array responds to a signal from any direction. By using an equally spaced linear array, the function to implement the steering vector is proportional to the sine of the angle, which makes implementing the algorithm very easy. Also, the MUSIC algorithm needs at least $N+1$ sensors to detect N sources, so by using four we can distinguish the two sources used in the Robosub competition, and having the one extra receiver improves the end result of our system.

The table below summarizes the design choices from this section.

Choice	Benefits	Trade-offs
Class D Push-Pull Amplifier	• Power Efficiency • Large voltage swing	• Worse with capacitive loads • Needs extra filtering
LC Matching Network Topology	• Low part count • Effective around center frequency • Easy to design	• Worse at frequencies farther from resonance • More Less filtering
Tiva MCU	• Available • Prior experience • Easy signal generation	• Semi-accurate PWM generation

Choice	Benefits	Trade-offs
STM32 MCU	• Affordable • Easier onboarding (compared to FPGA) • Low power consumption	• Slower than an FPGA • Included ADC module is slow for 4 channels
MUSIC DOA algorithm	• Simple to implement • Existing reference implementation in Matlab	• Breaks when correlated signals are present (echos in pool)
Four element Uniform Linear Array of hydrophones	• Above 3 sensor minimum for two sources • Mounting solution and hydrophones were already available	• No elevation estimate • Limited ability to perform spatial smoothing

6. Hardware Implementation

The transmitter module is built to drive a BT-2RCL-OSx piezoelectric transducer, which features a natural resonant frequency of approximately 28 kHz [9]. The PCB for the driver board can be seen in figure 6. The signal generation begins with a Texas Instruments Tiva C microcontroller, which outputs the target PWM carrier frequency between 25 kHz and 40 kHz and an enable pulse. To protect the sensitive digital logic from the high-voltage analog stages, the PWM signals pass through an ISO7720 Digital Isolator and an SI8241/8244 digital gate driver. The gate driver manages a Class D Push-Pull Half-Bridge amplifier built with discrete power MOSFETs. The gate driver is critical as it injects precise "dead time" between the inverted PWM signals, ensuring that both MOSFETs are never open simultaneously, which would short the power rails.

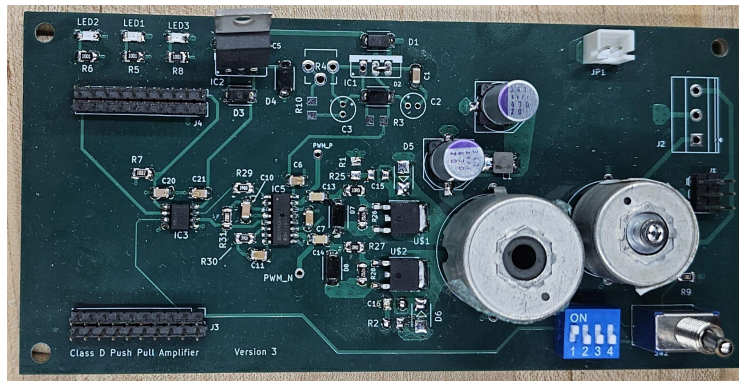


Figure 6: PCB for transmitter driver board. MCU mounts on the left side, and the output is taken from the right side.

The amplifier's output is stepped up via a 1:6 transformer to achieve the high voltages necessary for the transducer. Because the transducer inherently possesses a static parallel capacitance of 14.37 nF, as seen in figure 7 which shows the measured parasitics of the actual transducer, driving it directly with a square wave is highly inefficient. To physically match our signal to our transducer, we had to use 1mH series inductors. The value for the series inductor is a massive tradeoff for how well the signal looks at the edge frequencies, 25kHz and 40kHz. Currently, we use 1mH which resonates with the transducer at 30kHz and gives the best overall compromise for all transmission frequencies.

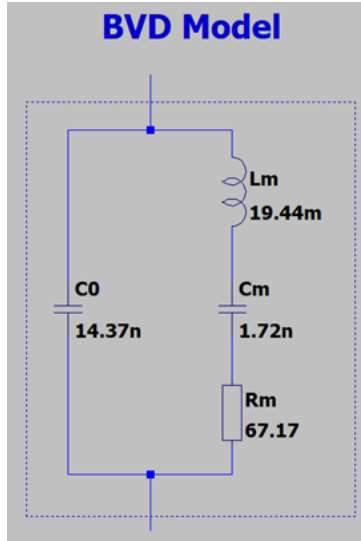


Figure 7: BVD Model of our transducer for the transmitter, the BT-2RCL-OSx. Values were determined in the lab using an LCR meter.

The transmitter circuit was created on a custom printed circuit board, which underwent multiple revisions to improve performance and stability. Version 3 of the board was upgraded to a four-layer design to minimize Electromagnetic Interference (EMI) and high-frequency noise. A continuous ground plane was implemented on layer two, and massive copper pours were routed around the high-current transformer primaries and MOSFET drains to reduce return loop inductance. Additionally, the physical layout was condensed to place the decoupling capacitors, gate drivers, MOSFETs, and snubbers in tighter proximity. The board integrates a physical DIP switch for one-hot frequency selection and screw-down terminals to securely mount the transducer. The entire driver board PCB, with the Tiva MCU attached to it, along with the DC-DC power module, LiPo battery, and transducer can be seen wired together in figure 8 below.

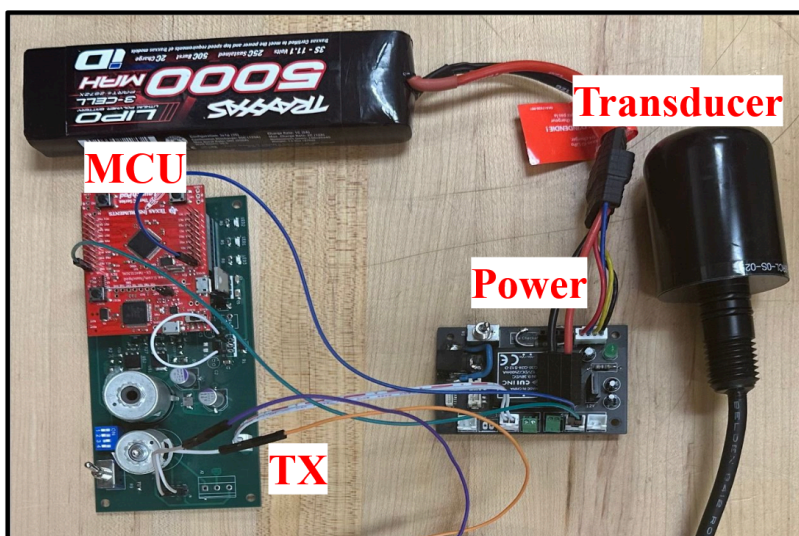


Figure 8: The entire setup for our transmitter, with the driver board and MCU wired to the transducer, and being powered by the DC-DC module and LiPo battery pack.

Receiver

The receiver hardware is composed of four individual Low Noise Amplifier (LNA) boards, mapping to an array of four hydrophones. The hydrophones are either hand made ones designed for 200 kHz, or BTECH 1201 models. Each LNA board processes a differential twisted-pair signal through several analog stages before reaching the STM32 F446RE's ADC. The initial stage utilizes an INA217 instrumentation amplifier, providing 20x (~26 dB) of gain. This is followed by a secondary fixed 30 dB gain stage, elevating the microvolt-level acoustic signals to approximately 4 V_{pp}. The next stage is a 2nd order Sallen-Key bandpass filter that uses an ADA4841-1 op amp. This is a high Q factor configuration, however, it proved to be too unstable for our board so the passive network was adjusted, reducing the filtering ability, but making the circuit stable. After that, a VCA810 voltage-controlled amplifier is used to dynamically attenuate or amplify the signal to prevent ADC clipping. The analog control voltage for the VCA is provided by an onboard AD7801 DAC, which receives its parameters from the STM32 via an SPI bus. Before going to the ADC, an 8-th order bandpass filter using the LTC1562 IC was implemented to further remove and out-of-band noise, including the switching noise introduced from our DC-DC power supply, and a level shifting circuit was introduced to move the AC signal up 1.5VDC to prepare it for the STM32's 0-3V ADC. Figure 9 shows a close-up of the LNA board, with the hydrophone input on the right, and power supplied on the left. The output is taken by the purple jumper wire, and the rainbow braid of five jumper wires carries the SPI signal for the VCA. Figure 10 shows the full setup with all four receiver channels, the STM32, the DC-DC module, and the LiPo batteries.

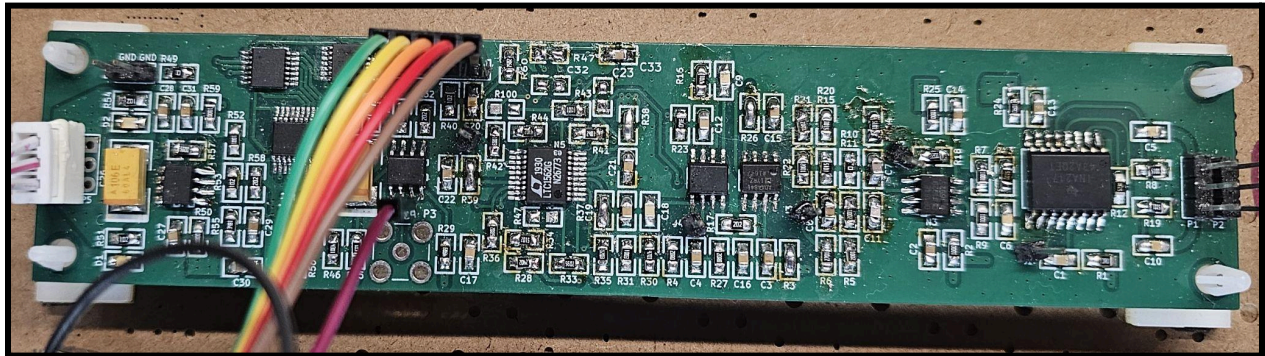


Figure 9: Low Noise Amplifier (LNA) Board.

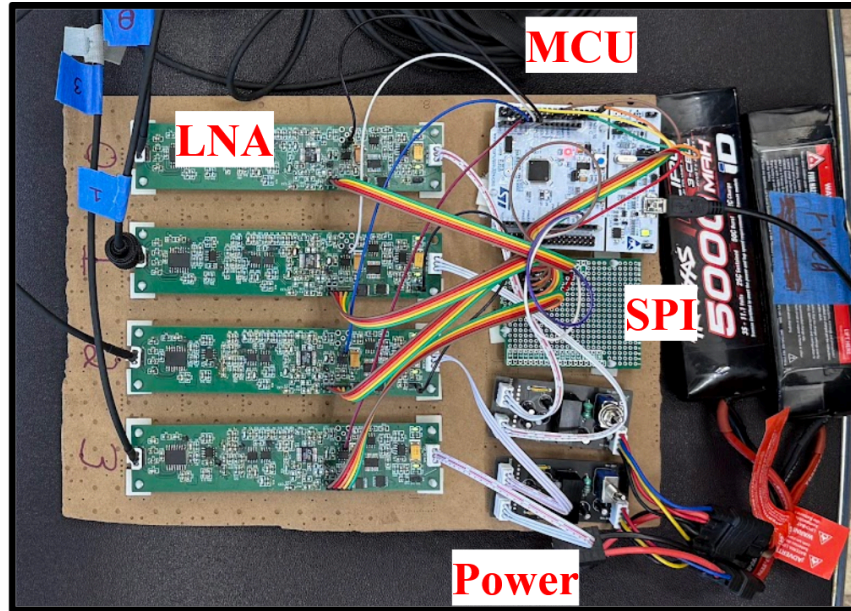


Figure 10: Full receiver module setup.

7. Software & Control

Overview of Direction Finding System Software

The direction finding system consists of three distinct components : a C++ implementation of the MUSIC DOA algorithm, a state machine based STM32 firmware project capable of executing a variety of commands, and a Python script that can send and receive commands from the MCU, accelerating system testing and providing an example of how the receiver system could be integrated into the larger drone system. The firmware was designed to target, and tested using, an STM32 F446RE development board. The code should be portable to any STMF4XX chip, assuming pin assignments are modified appropriately. The pin assignments used are detailed in the appendix. Development was done on an Ubuntu 24.04.4 installation, with STM Cube IDE version 1.19. If using newer versions of STM Cube IDE (≥ 1.20), modifications must be made to the project structure. Consult STMs updated Cube IDE documentation if using an updated version of the software.

Implementation of MUSIC Algorithm

The MUSIC algorithm was modeled off of the Matlab implementation [6]. The C++ implementation utilizes the Eigen C++ library. This library is a header only library that provides several important functions required for implementing the MUSIC algorithm (robust vector and matrix operations/transforms, eigen value decomposition functionality, and a Fast Fourier Transform implementation).

The C++ implementation encapsulates the necessary helper functions inside of a class named DOA. This class contains several constructors that allow different parameters to be configured, such as the frequency to detect the arrival of, the speed of the wave in the medium (1480 m/s for the speed of sound in water), and the sampling frequency for each channel (400 KSP/s as tested).

The primary interface is the estimateDOA function, which accepts a pointer to an array containing the voltages measured on the hydrophones. This function processes the input data by converting the real valued voltages into complex values, and then subtracting the mean from the data. This is required as keeping the fixed 1.5 volt offset from the LNA board in the signal was found to cause issues with the algorithm. After the data is processed, the covariance matrix of the data is computed, and this matrix is passed to the helper function estimateDOA_cov. The estimateDOA_cov function was also utilized to prove that the C++ MUSIC implementation matched the output of the Matlab implementation when provided the same covariance matrix. The MUSIC branch of our code base contains more information about running the code, and testing infrastructure to compare the output of the C++ function to the Matlab implementation.

Calculating the DOA given the input data is performed identically to Matlab, with Eigen utilized in place of the built in Matlab functions. The primary difference from the Matlab implementation is incorporating the different delays that are present in the system. Maintaining an accurate phase relationship between the incoming signals is critical for the output of the MUSIC algorithm to be coherent. The hydrophones, LNA boards, and the ADC sampling itself add delays into the measured signal. Through lab testing, this delay value was calculated, and added into the steering vector during its construction (the variable board_delays in steeringVectorULA_delay contains the individual board delays, in radians). This was found to produce accurate results in our testing, however should be reevaluated in the event of major hardware modifications to the LNA board.

Control Logic

The STM32 firmware was designed around a UART based command protocol. In main.cpp, there is a primary while loop that dispatches different stages of a command, depending on the current state of the system. The state of the system is modified from three different sources - the return value of the executing command stage, the UART interrupt handler, and the ADC interrupt handler. The state transition logic gives first priority to a received command, second priority to the ADC handler state update, and last priority to the updated state from the return value of a function.

When this code is redesigned, a more robust state transition logic/main program logic could be worth implementing. Currently, there is a remote possibility of code malfunction, if an interrupt fires mid state transition logic. This has not been observed while testing, but is a possibility and could lead to illegal program states.

The high level control logic is demonstrated by the two figures below. The figure 11a demonstrates the logic deployed in main.cpp, and figure 11b shows the high level steps that are performed to execute a DOA calculation, in the event a “Start” command is received. When there is not a command actively being executed, the system waits in an idle state, performing no actions. For commands that require a “valid” ADC sample, the process for determining if a sample is valid is outlined in the appendix.

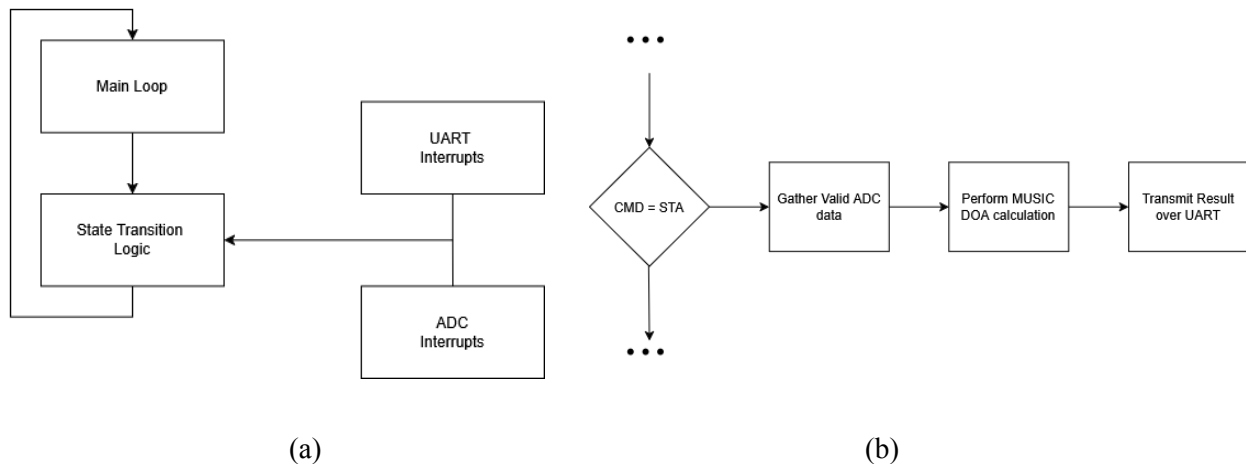


Figure 11: Main firmware logic

Example of command (DOA calculation) execution

The current list of supported commands is shown in table A2. These commands fit into 8 byte packets (although this could be expanded with minimal code changes), and when parsed by the MCU, the current state is updated and the command is executed. Before command execution begins, an acknowledgement command is sent in order for the host computer to verify that the system is currently operational (no ACK = system failure). Some commands such as the Set Frequency require no arguments and have no response, so the response stage in Fig. 12 is skipped. For other commands, such as Data Transfer, the command is sent with arguments, which are parsed by the MCU, and modify the size of the resulting response from the system.

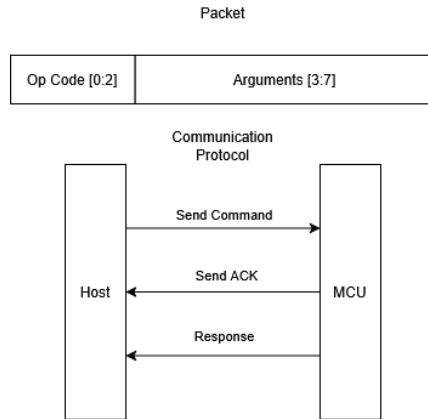


Figure 12: Illustration of command protocol used to communication with STM32 firmware

Utility Programs

The last software component developed was a Python utility to interact with the MCU. This script (`cmdTesting.py`), provides a simple command line interface that allows to type out a supported command, and provide the appropriate arguments. This utility will construct the command packet and send it over UART using the PySerial library. This application will then display the response. For data transfer commands, the program enables saving the received data into a file for future viewing, or plotting the voltages from each channel right away, allowing for quick verification of the test setup. The ADC data can then be visualized using `read_np.py`, which will plot the received ADC data, and can optionally plot the frequency spectrum for a channel.

System Operation

After the system is powered on, the hardware modules are instantiated on the STM32, and the design enters an idle state. The next step when performing a test is to either utilize the Calibration command, which will perform an automatic calibration of the gains for each hydrophone (equalizing the peak to peak voltages of each channel), or manually calibrate the gain for each board utilizing the SPI command. The details of the autocalibration logic can be found in the appendix.

The SPI gain control accepts values from 0 to 255 (8 bits wide), however, the maximum gain value occurs at $SPI = 180$. This per channel gain control is required as each of the LNA boards and hydrophones have different sensitivities, and even when all else is equal, the strength of the incoming signal would not be uniform across the array. Having equal amplitude along the channels is not a strict requirement for the algorithm to work correctly (the only requirement is the signal contains the correct frequency and phase offsets, and is not clipping), however, it does improve data visualization when plotting the received ADC data.

After calibration is complete, the target frequency for the MUSIC algorithm can be set using the Set Frequency command, if it differs from the default frequency specified in `params.h`. After the system is calibrated and the correct frequency is set, MUSIC DOA estimates can be streamed from the MCU using the Start command. These results can then be displayed by the `cmdTesting.py` script, or piped directly into the greater drone software ecosystem.

8. System Integration

Transmitter Hardware-software Interaction

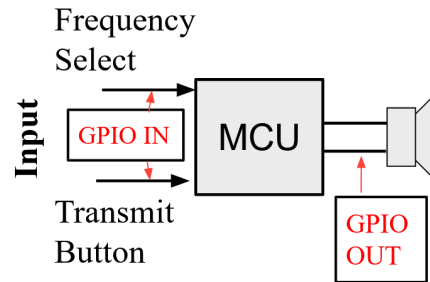


Figure 13: Diagram of hardware-software interaction for transmitter

The transmitter utilizes General Purpose Input Output (GPIO) pins to interface with the transducer. To control the output frequency of the signal, a DIP switch is utilized, and to begin transmitting a pulse, a standard button is used - both are wired to GPIO input pins on the MCU. GPIO output pins are used to generate the clock and gate signals required for the transducer to generate a pulse, the pulse width is configurable in the MCU firmware. The overall system integration is demonstrated in the figure above.

Receiver Hardware-software Interaction

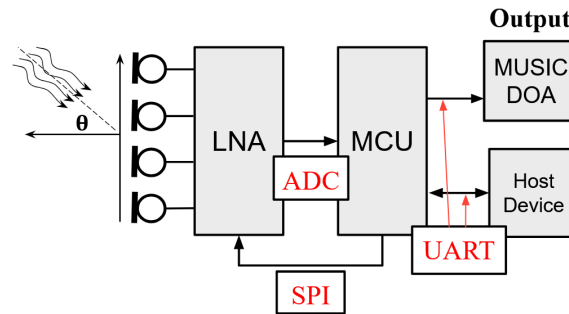


Figure 14: Diagram of hardware-software interaction for receiver

The receiver system interacts with the hardware using three distinct interfaces. The LNA boards have a variable gain output. This gain is controlled using a DAC on the board, and the values are loaded onto this DAC using a Serial Peripheral Interface (SPI) bus. There are currently four boards, one for each hydrophone, so this interface occupies six total GPIO pins on the STM32 (one chip select (cs) for each board, and a shared clock and data pin).

The voltage from the LNA boards is read into the STM32 using the onboard ADC module. The MCU is currently using the Direct Memory Access (DMA) feature of the onboard ADC to reduce the time it takes for the ADC values to become available in memory. The 2.4 MSPS ADC is multiplexed between the four channels, and driven by one of the onboard timers, resulting in an effective sampling rate of 400 KSPs for each channel. This is 5 times the Nyquist rate required to capture the fastest signal from the LNA board (40 KHz), however a faster external ADC module would allow for more flexibility with detecting the edge of an incoming wave.

Lastly, a Universal Asynchronous Receiver Transmitter connection is used to send and receive data between the MCU and an external system. The connection is configured to run at 921600 baud, and enables the command protocol outlined in the appendix to be successfully implemented.

9. Experimental Setup

The experimental setup was designed to evaluate the direction-finding system under controlled acoustic conditions while also identifying the practical limitations caused by the test environment and receiver hardware. Because the MUSIC direction-of-arrival algorithm assumes an approximate plane-wave propagation across the receiver array, both the physical environment and the receiver selection had a direct effect on the quality of the measured data. The goal of this section is therefore not only to describe where and how the system was tested, but also to explain how setup conditions influenced the reliability of the results reported in Section 10.

Testing Methodology

We tested the system in two different environments: an anechoic water tank and an Olympic-sized swimming pool. Additionally, we used two different receivers in our array: commercial BTech 1201 and handmade receivers we just denoted as “Lehigh 200 kHz.” The access to the receiver hardware was something that was out of our control, thus more thorough testing was made impossible. Both setups satisfied the far-field requirements of the MUSIC algorithm for our linear array receiving the selected frequency (30KHz) [7]. Further information about these requirements can be found in the appendix.

Water Tank:

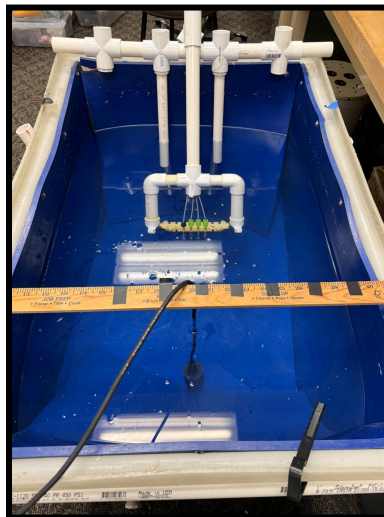


Figure 15. Water tank test setup.

The water tank provided a controlled and low noise environment where we could easily troubleshoot functionality using the oscilloscope. However, the limited space meant that we had to be careful about the distance the pinger was from the walls, surface, and bottom as well as our basic backstop consideration. Due to this confinement, our angular window was reduced [see results].

Pool:

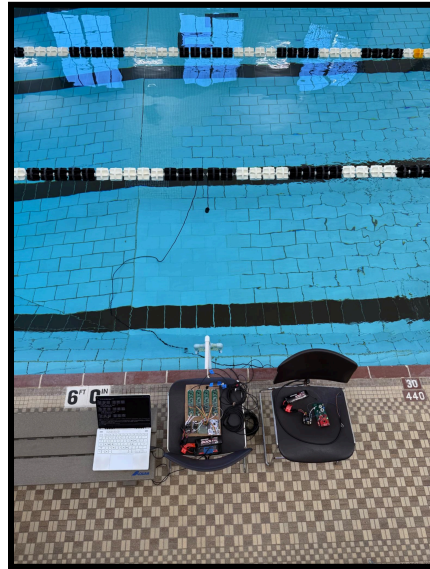


Figure 16. Pool test setup depicting all equipment

The RoboSub competition occurs in a pool, thus our results here are the most important. The pool eliminated most of the confinement concerns of the tank but was a noisier environment. The higher noise level combined with the more sensitive and directional Lehigh 200 kHz receivers yielded less accurate and consistent performance than the commercial receivers.

Receivers:

The handmade Lehigh 200 kHz receivers demonstrated they were much more directionally sensitive than the commercial BTECH 1201 receivers. The effect of this was that by rotating the Lehigh 200k receiver, you could get drastically different results (particularly in the perception of left vs right). Additionally, they proved to be much more sensitive to noise spikes in the pool environment which made the edge detection software much less effective. The commercial receivers proved to be much less directional and stable—they should be the preferred choice for deploying the system in any environment.

Measurement Setup

In each environment, we took physical measurements of the pinger's placement with respect to the receiver array (vertical and horizontal distances) to calculate the expected angle of arrival. Then we took measurements at each location using the STM32's firmware and exported the raw ADC data back to the host computer to confirm the results on the STM matched the Matlab implementation *musicdoa* [6]. At each location, for internal testing and for formal demonstration, we took at least 3 measurements per position to ensure consistency and repeatability.

In the tank, we used a yard stick to both measure and hold the pinger steady. Results to the nearest 0.5°.

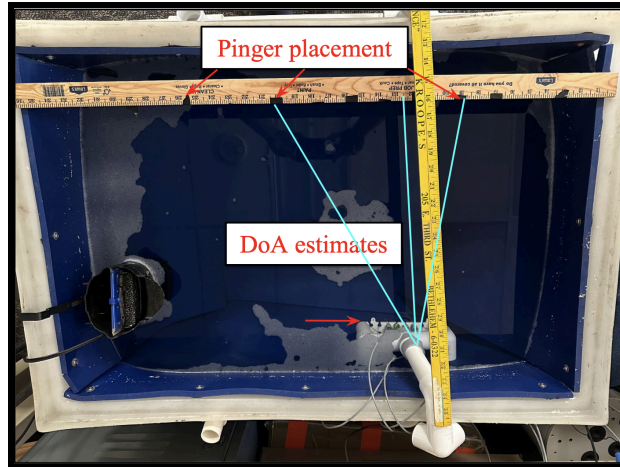


Figure 17. Tank test measurement methodology

In the pool, we used the markings on the side of the pool and the lane dividers to estimate the physical position of the array and the receiver. Each divider was 6" in length, so our precision was to the nearest 0.5'. Distance between rows was 7' (therefore the distance from the array to the first row was 7' - 10").

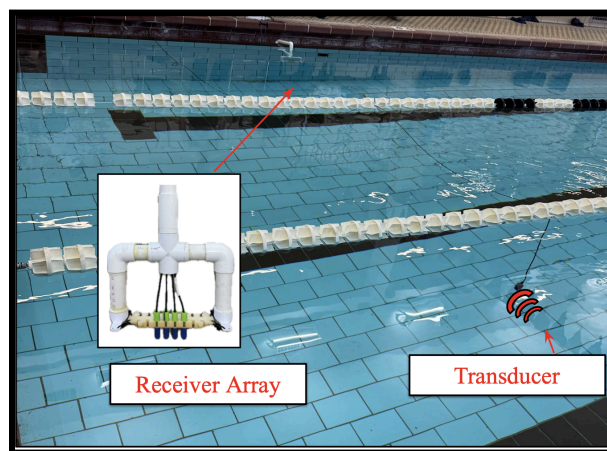


Figure 18. Pool test measurements based off the markings on the sides and the spacers on the lane dividers

Equipment used

An overview of the equipment used can be found in table A3.

10. Results & Performance

Transmitter Results

Our testing confirmed that the transmitter was able to generate sinusoidal figures for all four frequencies. Additionally, the amplitude was around 400Vpp for each frequency. It is clear the signals for 25 kHz, 35 kHz, and 40 kHz are much more distorted than the 30 kHz signal, however, the piezoelectric transducer has a very strong resonance at 28kHz and it may be too difficult in the future to get broadband low distortion outputs. Additionally, our testing proved that the LNA board has strong enough filters to remove the distortion on the transmitted signal for all four frequencies.

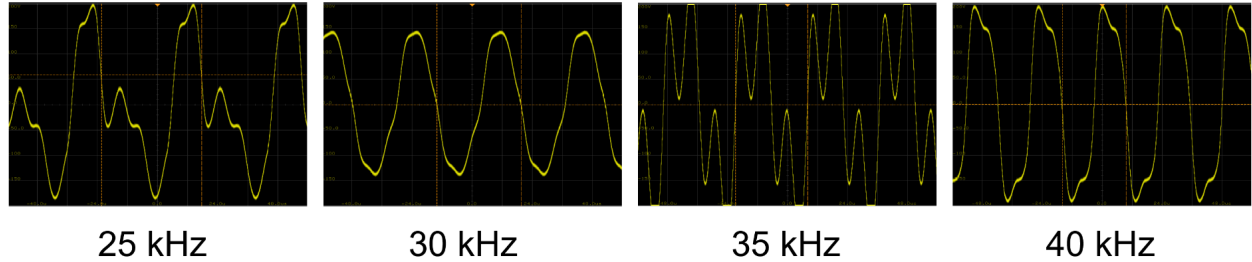


Figure 19: Output waveform across transducer for all four frequencies.

Overview of Receiver Results

The below section outlines the results in two separate testing environments : one of the pools within Taylor Gym, and the anechoic water tank (as shown in section 9). The testing performed differs from the proposed use case on an AUV, as the testing apparatus is mounted in close proximity to a reflective surface (the pool/tank walls). This adds additional complexity to the ADC data collection on the receiver end, and contributes to increased sensitivity of the system overall.

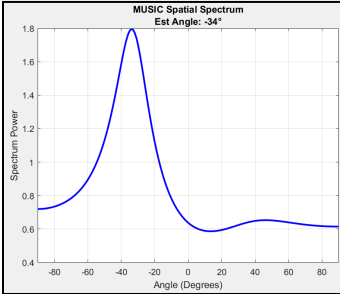
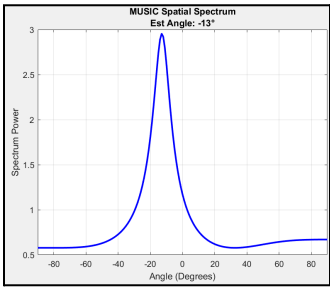
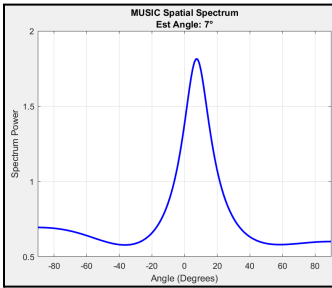
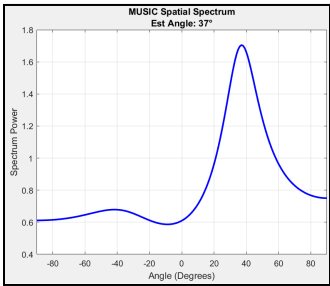
When testing the 200 kHz receivers in the pool, we found the results would vary significantly depending on the depth of the pinger. This was accompanied by additional inconsistencies (especially with noise). For this reason, we preferred the results from our limited testing with the BTech 1201 hydrophones. They seemed less directional, but most importantly they were much more consistent than the handmade 200 kHz receivers. Additionally, we believe that the geometry of the test setup (a close backstop) makes this the worst case scenario, as the array will be eventually mounted to an AUV and be situated in a more open environment (like the middle of the pool).

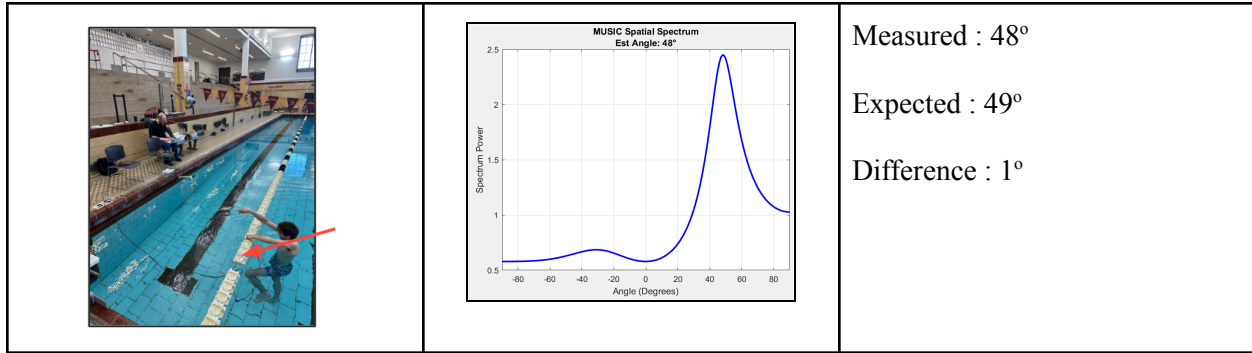
Pool Test

The following results are from our best pool test on April 14th 2026 (one of only two tests we got using the commercial receivers), the middle column contains the DOA spectrum generated by the recorded ADC data. When the algorithm works correctly, there is one distinct peak per signal source in the testing environment, with the location of the peak corresponding to the direction of arrival estimate. For our testing with a single transmitter, there is a single peak, however for the Robosub competition which has two transmitters, there will be two strong peaks.

These spectrums were generated on an external computer using Matlab, however the results are identical to the ones generated on the MCU. The spectrum was calculated externally for convenience, as the generated spectrum on the MCU is not intended to be used outside of the MUSIC implementation.

Table 1: Pool Test Results (BTech 1201 Receivers)

Testing Configuration	DOA Spectrum	Result
		Measured : -34° Expected : -45° Difference : 11°
		Measured : -13° Expected : -27° Difference : 14°
		Measured : 7° Expected : 0° Difference : 7°
		Measured : 37° Expected : 40° Difference : 3°



Anechoic Tank Test

These results were obtained from an anechoic tank test performed on April 29th 2026. One difference between previous tank tests was that the hydrophone array was mounted at a 45° angle, as shown in Fig. 20. This was found to improve the consistency of the results (Note: This was using the 200 kHz receivers).

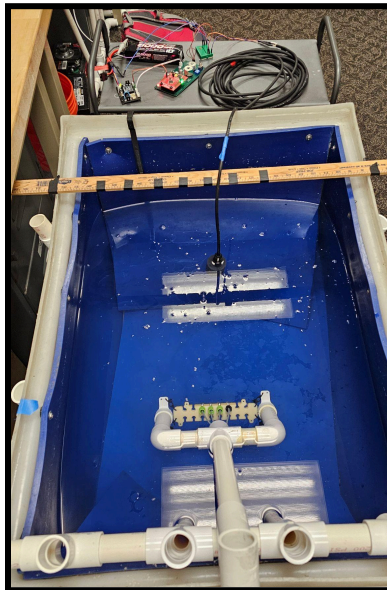


Figure 20: Setup used in the anechoic tank, with the PVC mount for the receiver's ULA and the transmitter dangled on the other end using a meter stick.

Table 2 demonstrates the DOA estimations from 25 ADC data samples acquired using the testing setup shown above. The cell highlighted in red contains an outlier result. After the redesign of the ADC system, these results are rare, but still possible. While these results were calculated off device (using the recorded data from the MCU), they were found to be consistent with the output of the “Start” command on the MCU, which was used to confirm the approximate location before recording the ADC samples for future testing and debugging purposes. Additional results and a video demonstration can be found in our groups weekly progress report 13 [8].

Table 2 : Tank Test Results (200 kHz Receivers)							
Position (inch marker)	Measurement (degrees)					Avg DOA (degrees)	Theoretical DOA (degrees)
14	-28	-29	-28	-31	-30	-29.2	-22
17	-14	-15	-14	-13	-13	-13.8	-11
20	-23	-4	-6	-6	-6	-5.5	0
23	3	5	5	4	4	4.2	11
26	16	14	14	14	14	14.4	22

Discussion of Results

In both testing environments, the system performed within the target accuracy constraints of $\pm 15^\circ$ of the actual incoming signal. While steps were taken to ensure experiment consistency, the experimental setups inherently include some variations that were unavoidable. For the pool test, the floor had a steep slope downwards on the left side of the testing environment, which could have been a contributing factor to the degraded performance of the system on the left side. For the anechoic water tank test, the transmitter was held in place by hand, which adds some variation in the amount of possible vibrations/microadjustments to the transducer's position.

While testing the system with the 200 kHz receivers, it was also observed that the system is sensitive to the depth and orientation of the transducer that is sending the pulse. In the pool, varying the height of the transducer could produce results that were read as being on opposite sides of the pool, and in the tank, modifying the height of the transducer was found to produce $>20^\circ$ swings in the estimated DOA, if any estimate was present in the resulting spectrum. It is believed that these inconsistencies will be improved by transitioning the design onto the LUR AUV (less reflections will be present on the input), and by incorporating the pinger system into a more fixed, water tight enclosure. This behavior was only observed using the 200 kHz receivers—not the commercial BTech 1201. However, the exact cause of this specific issue is still currently unresolved as we weren't able to conduct more thorough tests with the commercial receivers (Dr. Zheng needed them back). Thus, it could have been some other instability in our test setup or system; the aforementioned possible solutions are just speculation.

11. Function Verification

Our system successfully performs the following functions :

1. Transmits a pulse...

- a. Pulse is under 10ms in duration.
- b. At each of the following frequencies: 25kHz, 30kHz, 35kHz, 40kHz.
 - i. Stronger than 100V RMS (equivalent to 300Vpk-pk)



Figure 21 : Oscilloscope result showing our received pulse is just under 10 ms in duration

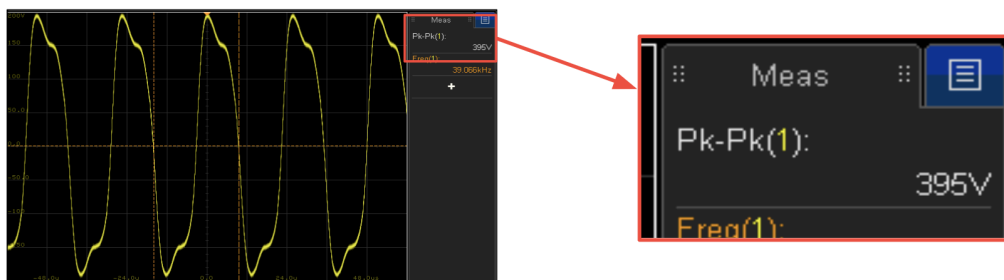


Figure 22 : Oscilloscope screenshot of a 30 KHz wave transmitted by the transducer

Fig. 21 demonstrates that the pulse duration is under 10ms. Figure 22 demonstrates how the pulse transmitted by the transducer is at the correct frequency, and above our 100Vpk-pk target. The results for other frequencies are shown in Fig. 19. These scope screenshots demonstrate that across all four target frequencies, the output voltage is above 100Vpk-pk, and the frequency is accurate. It is possible that our pulse exceeds 400Vpk-pk, however due to measurement equipment limitations, this was unable to be measured.

2. LNA Receives and processes the transmitted signal...

- At each frequency.
- Controllable Gain of signal between 0-3Vpk-pk.

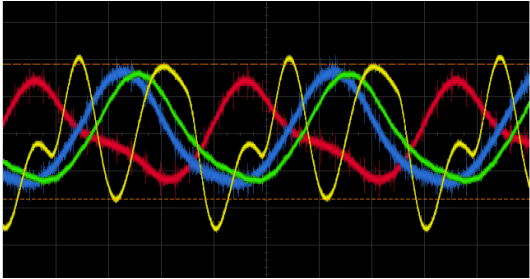
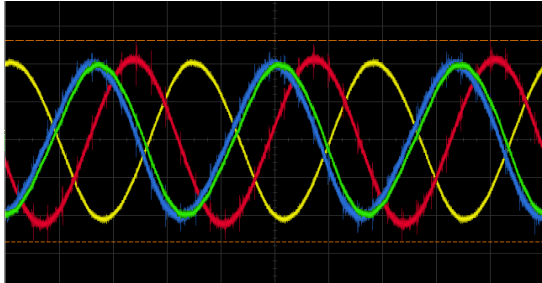
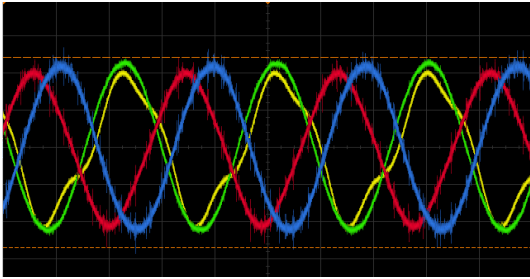
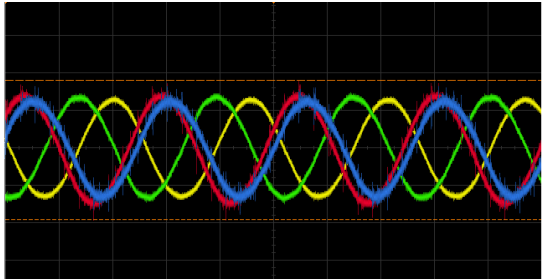
Table 3 : Oscilloscope Screenshots of LNA Board Output			
			
25 kHz		30 kHz	
			
35 kHz		40 kHz	

Table 3 shows how the transmitted signals from the transducer are correctly processed by the LNA boards. Additionally, the gain of the boards can be modified through the use of the “SPI” command implemented through the STM32’s firmware. This is demonstrated in Fig. 23.

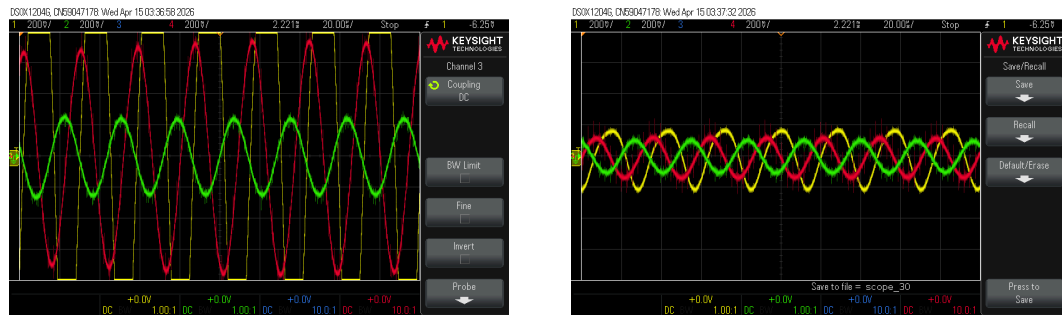


Figure 23: (i) LNA board output before SPI control

(ii) LNA board output after SPI control

3. STM32 ADC samples the output of LNA...

- Has a ≥ 20 dB peak at the specified frequency.

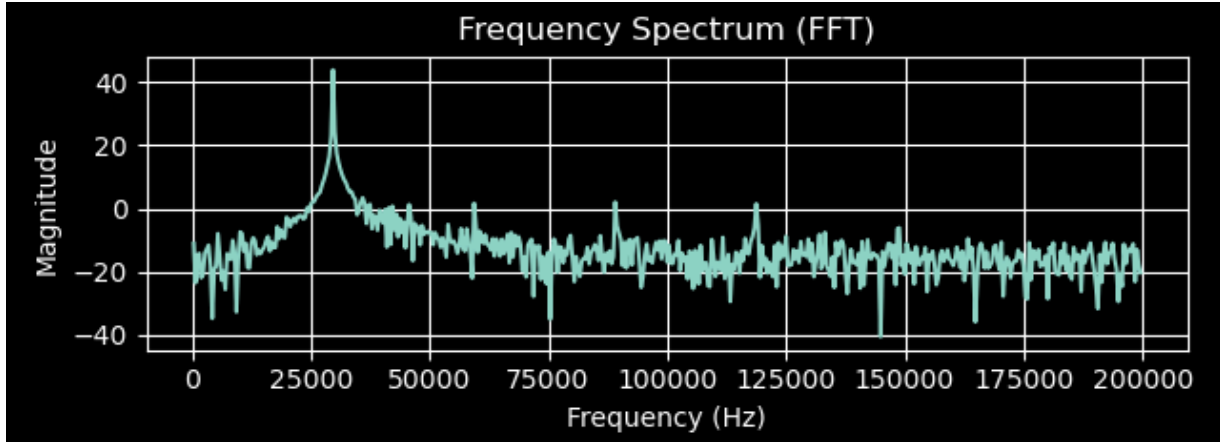


Figure 24: FFT results from ADC taken from LNA board on host computer

Figure 24 shows the frequency spectrum of one of the hydrophones. This was generated with exported ADC data from the STM32, during one of the tank tests. This spectrum demonstrates that the captured signal has a strong 40dB peak at the incoming frequency (30 KHz), with minimal harmonics in the spectrum. At other frequencies (25 KHz especially), the harmonics are more apparent, however these don't interfere with the MUSIC algorithm.

4. STM32 implements the MUSIC algorithm...

- a. Output is correct to within 15° for 30kHz signal.

The performance of the MUSIC algorithm on the STM32 is demonstrated in table 2. While the results from that table were calculated off device, the results have been shown to match those that are generated by the STM32. In a pool testing environment, the system was also within these accuracy constraints, as shown in table 1.

12. Impact & Applications

The primary impact of this project is its contribution to the Lehigh Underwater Robotics team. A functional underwater direction-finding system is required for two pinger-based RoboSub tasks, making it an essential step toward the team's broader autonomous capability. In the RoboSub handbook, pingers are used to guide the AUV toward specific tasks, and the scoring section identifies two pinger tasks with additional points available for random pinger selection [2]. In addition to the receiver array and direction-finding system, the customizable acoustic pinger developed for this project gives the team a repeatable way to test and validate future designs. Together, these systems provide both an immediate competition benefit and a foundation for continued development.

Our documentation, including this report, is also intended to support future development of the system. Familiarity with the system is necessary for it to be expanded into something that can be deployed on the drone. In future versions, the host computer will likely be replaced with the onboard Jetson, and the direction estimate will need to be integrated with the vehicle's autonomy stack. Details about the test setup, receiver choice, and results are included so that future team members can understand which design choices were successful and which limitations still need to be addressed.

This documentation also reduces the gap between a successful experimental prototype and a deployable vehicle subsystem. At the end of this project, the system is not yet a fully integrated autonomous navigation module on the AUV; however, the major technical risks have been identified more clearly. The experimental results show which receiver configuration was more reliable, what environmental conditions affected edge detection, and what signal-processing assumptions were most sensitive during testing. This gives future LUR members a more focused development path: replacing the host computer with the onboard Jetson, packaging the receiver array for vehicle mounting, improving real-time edge detection, and integrating the estimated pinger direction into the vehicle's autonomy stack. In this sense, the impact of the project is not only the final hardware and software produced, but also the narrowing of the remaining design problem for future teams.

Beyond its direct use by LUR, underwater acoustic direction finding has several real-world applications. Since radio-frequency signals attenuate quickly in water, underwater vehicles often rely on acoustic signals for navigation, communication, and localization. Reviews of AUV localization describe acoustic methods as central to underwater navigation because GPS and many terrestrial localization methods are not directly usable underwater. AUVs that can determine the direction of arrival of an acoustic signal can navigate toward marked underwater objects, locate deployed sensors, or assist in search and recovery tasks. For example, acoustic pingers can be used to mark underwater equipment, recovery targets, or navigation references. Underwater locator beacons are also used on flight recorders, where they emit acoustic pulses after water immersion to aid recovery. Similar localization methods are relevant to marine robotics, underwater inspection, search and recovery, environmental monitoring, and biological research involving underwater acoustic sources. [5]

Overall, the project has both immediate and long-term value. In the short term, it gives LUR a tested direction-finding system and a customizable pinger that can support RoboSub preparation. In the long term, it gives future members a documented platform that can be improved, integrated, and adapted as the AUV becomes more capable. The broader significance is that the project demonstrates a practical

implementation of underwater acoustic localization, a capability that is widely useful whenever autonomous systems must navigate or find targets in environments where vision, GPS, and radio-frequency communication are limited.

13. Conclusion

In conclusion, our final system can:

- Transmit 25-40kHz acoustic signals in an underwater environment
- Capture ADC data and perform DOA calculations, transmitting the result over a command based communication protocol
- Produce completely correct DOA calculations on simulated data, and with 15° accuracy in a pool and tank environment

The system deploys a correct C++ implementation of the MUSIC algorithm that is able to transmit one DOA estimate to the host system every 10 ms. While the system was designed using an STM32 F446RE MCU, the code is able to be ported to other STM32 chips with minor modifications, allowing the design to pivot an STM32 F411 development board, or implement a customized solution that is integrated with the LNA boards in order to save space in the final design.

The individual components of the system have been tested rigorously, however work remains to be done on overall system reliability. Certain intermittent problems, such as the SPI gain control behaving inconsistently have been resolved, but certain problems such as the systems performance when the transducer is moved to a different height remain open.

Our project provides the groundwork required to implement direction finding capabilities onto the Lehigh Underwater Robotics team. To achieve this, the receiver system needs to be integrated into the existing electronics infrastructure, and the array needs to be redesigned so that it can be mounted to the front of the drone's frame. For the transmitter, in order to test the system in a wider variety of positions, the electronics for the pinger need to be enclosed in a water tight vessel. The components to achieve this were ordered and are currently available, however, due to time constraints this integration was not yet performed.

Overall, our system fulfills the design requirements and can successfully determine the direction of arrival of an incoming acoustic signal. This signal is generated by a custom pinger design that allows for the pulse characteristics to be modified, allowing for greater flexibility when testing the functionality of the receiver system. Through the documentation this project generated, it should be able to be used as a foundation for future systems that require DOA estimation.

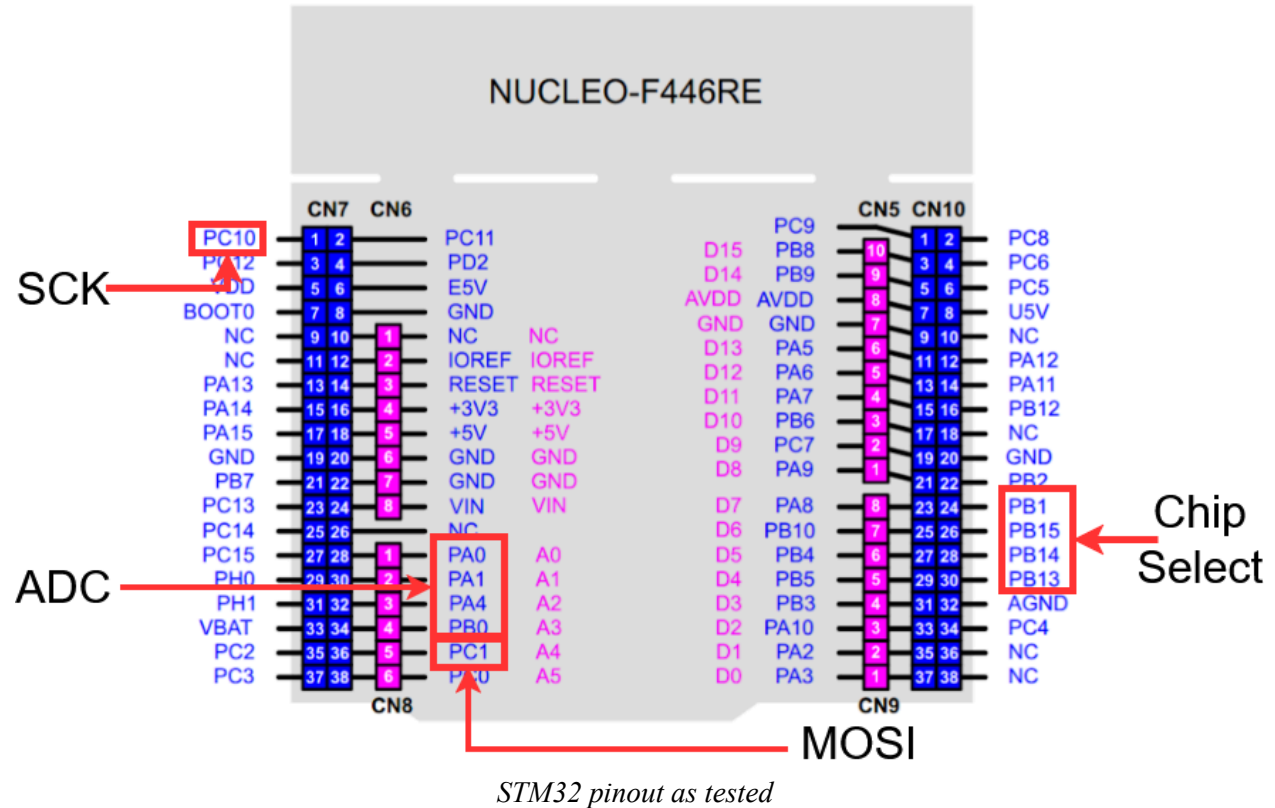
14. References

- [1] J. González-García, A. Gómez-Espinosa, E. Cuan-Urquizo, L. G. García-Valdovinos, T. Salgado-Jiménez, and J. A. E. Cabello, “Autonomous Underwater Vehicles: Localization, Navigation, and Communication for Collaborative Missions,” *Applied Sciences*, vol. 10, no. 4, p. 1256, Feb. 2020, doi: <https://doi.org/10.3390/app10041256>.
- [2] RoboSub 2025 Team Handbook, RoboNation Inc., Arlington, VA, USA, 2025.
- [3] Schmidt, R.O, “Multiple EMitter Location and Signal Parameter Estimation”, IEEE Trans. Antennas Propagation, Vol. AP-32 (March 1986), pp. 276-280.
- [4] STM, STM32F446xC/E, Feb. 2015 [Revised Feb. 2025]
- [5] G. Jekaterýńczuk and Z. Piotrowski, “A Survey of Sound Source Localization and Detection Methods and Their Applications,” *Sensors*, vol. 24, no. 1, pp. 68–68, Dec. 2023, doi: <https://doi.org/10.3390/s24010068>.
- [6] “Estimate DOA of Multiple Signals Using MUSIC,” *Mathworks.com*, 2016. <https://www.mathworks.com/help/phased/ref/musicdoa.html>
- [7] “Phased Array Antenna Patterns—Part 1: Linear Array Beam Characteristics and Array Factor | Analog Devices,” *Analog.com*, 2024. <https://www.analog.com/en/resources/analog-dialogue/articles/phased-array-antenna-patterns-part1.html>
- [8] “Weekly Progress Report 13”, Team A5, 2026, <https://docs.google.com/document/d/15Pb66DleO-W6VmioVYPyTGN0MNntWlfu/edit?usp=sharing&ouid=114264468429849944089&rtpof=true&sd=true>
- [9] “BT-2RCL-OSX.” *Company Profile*, www.btechacoustics.com/products/bt-2rcl-osx. Accessed 19 Sept. 2025. <https://www.btechacoustics.com/products/bt-2rcl-osx>
- [10] J. Hertz, “H-bridge DC Motor Control Using Complementary PWM, Shoot-through, and Dead-time - Technical Articles,” Aug. 29, 2022. Available: <https://www.allaboutcircuits.com/technical-articles/h-bridge-dc-motor-control-complementary-pulse-width-modulation-pwm-shoot-through-dead-time-pwm/>
- [11] L. Balogh, “Fundamentals of MOSFET and IGBT Gate Driver Circuits Application Report Fundamentals of MOSFET and IGBT Gate Driver Circuits,” 2017. Available: <https://www.ti.com/lit/ml/slua618a/slua618a.pdf>
- [12] G. Choi, “3.56 MHz, Class D Push-Pull, 2KW RF Generator with Microsemi DRF1300 Power MOSFET Hybrid,” 2011. Accessed: May 13, 2026. [Online]. Available: <https://ww1.microchip.com/downloads/en/Appnotes/1812%20C.pdf>

15. Appendix

STM32 F446RE Pinout

Due to issues with timing when setting the SPI values on the LNA boards, the values needed to be sent individually using GPIO pins on the STM32. This allowed for a slower clock rate than the official SPI modules allowed for.



Supported Firmware Commands

The below table outlines the currently implemented commands. The command parsing logic is split between data_transfer.cpp and user_includes.h.

Table A1 : Supported Commands			
Packet Type (op code)	Description	Arguments	Response
Acknowledge (ACK)	Acknowledge received command	N/A	N/A
Calibrate (CAL)	Calibrate gain of hydrophone array (currently buggy)	N/A	"FIN\0\0\0\0"
Set calibration low (SCL)	Set low threshold for calibration algorithm * consider redesign	Four byte UINT value	N/A

Set calibration high (SCH)	Set high threshold for calibration algorithm	Four byte UINT value	N/A
Start DOA estimate (STA)	Start performing DOA calculations, continuously calculates and transmits results until interrupted by another command	N/A	“RES[res1][res2]...” Where resX is a two byte little endian short containing the DOA estimate for source 1, 2, ...
Data transfer (DAT)	Collect and transmit ADC samples (converted into voltage equivalent) to host device	transmit length (little endian four byte UINT) - Number of samples to transmit per hydrophone after valid (unsigned char) - Number of samples after valid sample to transmit	Transmit ADC voltage in a stream [channel 0, channel 1, channel 2, channel 3, channel 0, ...] Values are four byte floats
Send SPI value (SPI)	Send SPI value over channel	SPI value (unsigned char) - 8 bit value to send SPI channel (unsigned channel) - Channel number (0 through 3) to transmit over	N/A
Set frequency (SFR)	Set the frequency of target signal, refresh MUSIC DOA object	N four byte UINTS : Frequency specified for each signal source * Can / should be changed to have a single four byte argument for the target frequency, which is then distributed across the frequency array	N/A
Set Noise Level (SNL)	Set threshold for what constitutes a valid ADC signal (variance of recorded signal)	Four byte UINT value	N/A
Stop (STP)	Stop current command * Consider removing, all other commands will interrupt current command already	N/A	N/A
Set MUSIC Buffer (SMB)	Set buffer size for DOA estimates * mostly untested (only 1 was used) and likely not necessary	buffer size (unsigned char) - 0 = buffer size of 1, 1 = 2, etc	N/A
Set MUSIC spacing (SPA)	Set array spacing in terms of wavelengths	spacing (float)	N/A

	* consider removing. New MUSIC constructors calculates this value based on C speed and distance (hardcoded, physical property of hydrophone array)		
--	--	--	--

ADC Sample Validity and Edge Detection

One of the major software hurdles, that remains a potential point of improvement, is how to determine if an incoming ADC sample is valid. In order for the MUSIC algorithm to produce correct results, it is imperative that the incoming signal does not contain any correlated elements. In the case of underwater acoustic signals, these correlated elements manifest themselves as echoes in the signal, caused by the reflections in the testing environment.

Currently, the firmware uses the `ADC_manager` class (defined in `ADC_functions.h`) to determine if an ADC sample is valid. This class uses the singleton design pattern (may be worth redesigning as a namespace and setting global variables in the namespace rather than managing a class instance), and provides a `gather()` function.

In order to collect a sample using this interface, call the `configure` method on an active instance of the class. This allows the pointer to the destination array, number of sensor samples to read, number of samples after a valid signal to collect the data from, and the next state after the data is gathered to be passed into the class. This was designed this way in order to simplify the main control logic of the code, which previously required adding a waiting state into each command that required an ADC sample.

Once the `ADC_manager` is configured for the sample, `ADC_manager.gather()` can be called. This will then wait for the ADC conversion complete callback to finish. The collected ADC sample is then validated. The current logic for determining if a sample is valid is outlined in the below figure. This is used by the `ADC_manager` class in order to develop a primitive edge detector, allowing the system to wait for a signal to appear in the ADC buffer before capturing the ADC data.

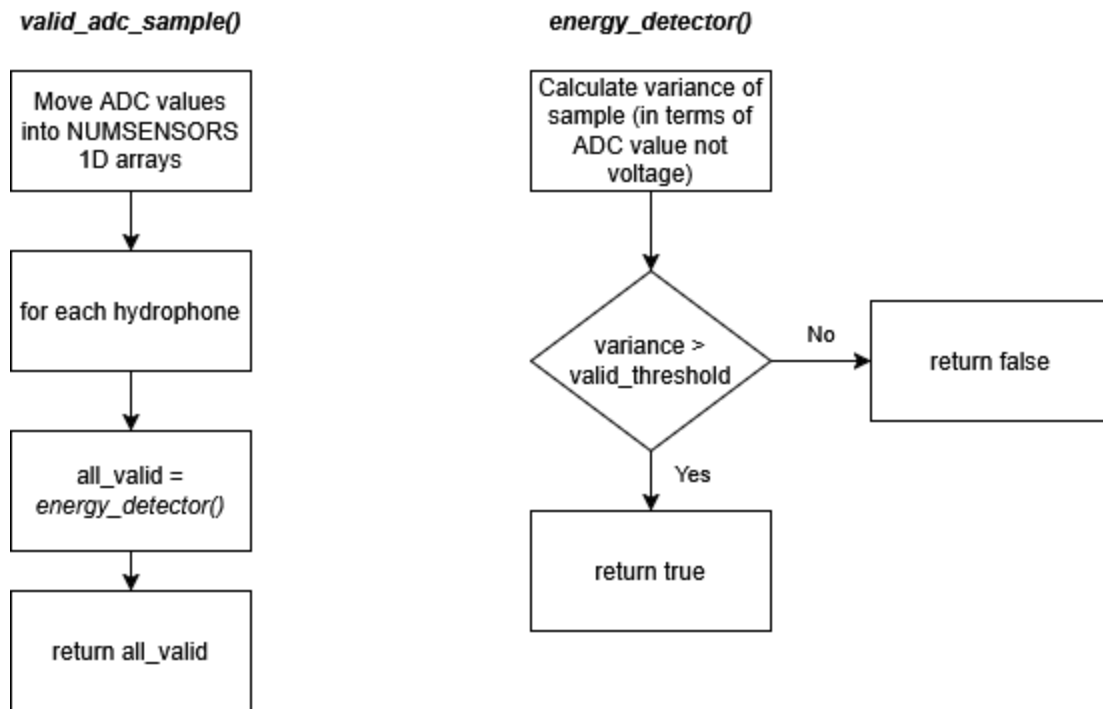


Figure A1 : *Logic used to determine if an ADC sample is valid or not*

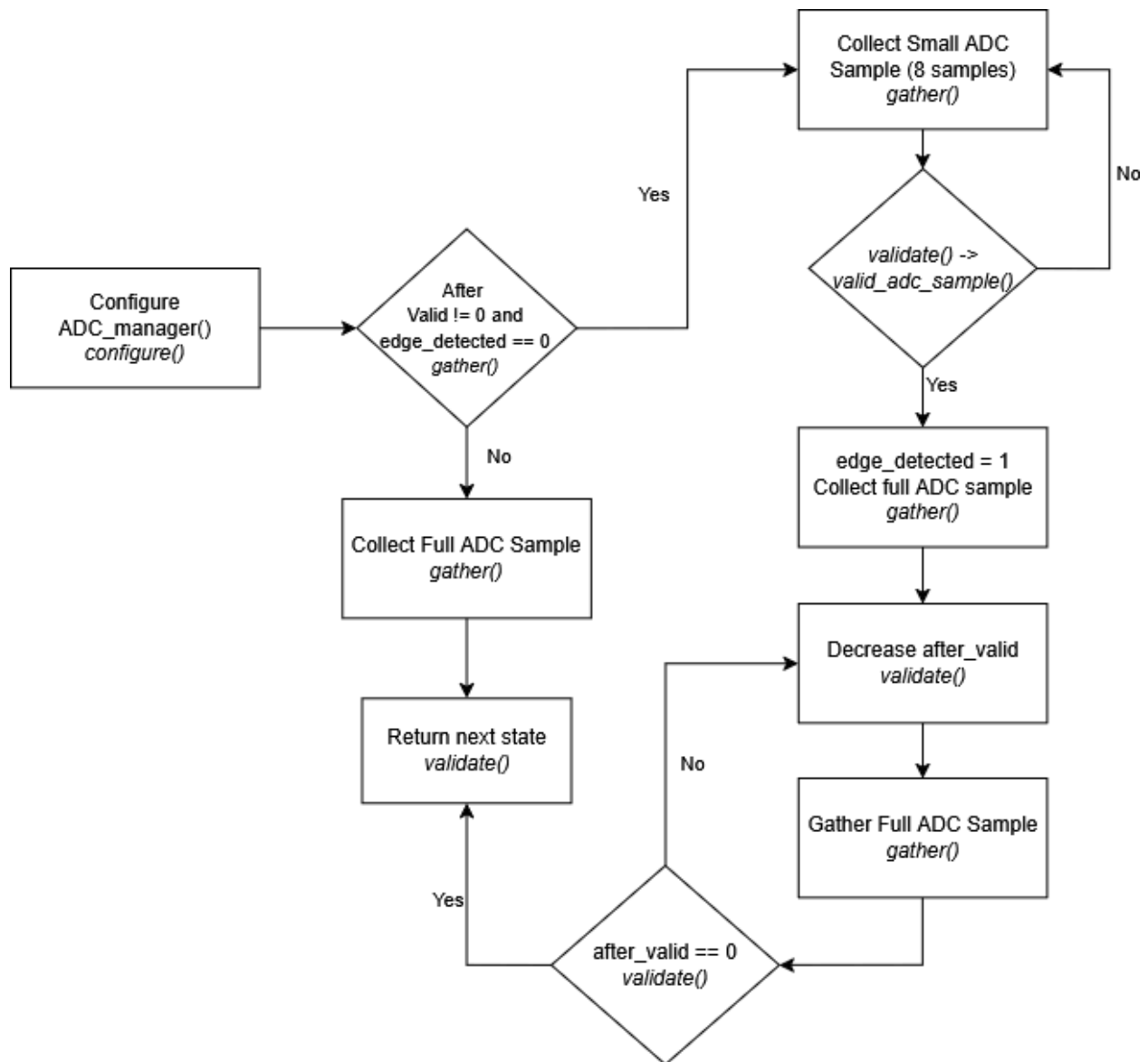


Figure A2: *ADC_manager class logic/usage*

Possible Improvements

This approach has a noticeable latency when collecting ADC samples due to the large amount of overhead present in the additional logic/function calls. This was implemented as it was a straightforward solution not because it was optimal. When this system is redesigned, a similar interface can be maintained, however the *validate()* function call should be developed further.

One possible solution is to use comparators on the STM32 to create an enable signal for the timer that drives the ADC (currently timer 8). The hysteresis of a schmitt trigger could enable triggering when the signal from the LNA board crosses a specific peak voltage, and this would completely remove the overhead of determining if the sample is valid in software.

The downside of this solution is that it could be more sensitive to noise spikes on the ADC input, however, the software solution is also currently sensitive to this. Furthermore, if the SPI values are

properly configured, the noise spikes should be lower in magnitude and if properly configured should not activate the comparator.

The software valid check does not look for clipped signals. This is not a major problem, however it is something that might be worth investing in the future to increase the robustness of the system. Lastly, there is a potential infinite loop in the validate logic. If the SPI values are all set to zero (meaning that the LNA board output signals will be held at a fixed 1.5 volts), if an ADC sample is gathered, the system will continuously check for a valid signal that will never come. This happens infrequently, however is something that should be addressed.

One potential solution that is currently untested is re-enabling the UART receive at the start of the ADC_v state logic in the main loop. This should enable incoming commands to override the data collection logic, allowing a hanging ADC_gather() to be interrupted, and the SPI values to be increased.

In conclusion, this entire component of the ADC software should be re-evaluated, with a focus on decreasing the amount of software overhead required to collect a valid signal. It currently works, however, is not an optimal solution for detecting an edge in an ADC signal (which is a requirement for the DOA algorithm to work correctly).

Firmware Automatic Gain Calibration

Current Status

After redesigning the ADC pipeline and replacing ADC calls with the ADC_manager class, this functionality has been encountering some difficulties. The current root cause is believed to be the pulse duration of the transmitter currently. Previous successful testing utilized a commercial pinger, with a longer (~500 ms) pulse width. The logic was to wait for the first valid sample, and then perform the automatic calibration function on subsequent samples. Currently, due in part to the shorter pulse width (10 ms) of the signal, and slower ADC response times when acquiring the samples, this algorithm is not functioning correctly, and needs to be reexamined in the future. The algorithm itself is functional and has been verified, the issue is with how the data is getting to the function, and how subsequent samples are acquired.

Gain Control Algorithm Pseudocode

The following algorithm is incorporated in the `gain_control` class, which is defined in `gain_control.cpp`.

Require: $spi_vals[NUMSENSORS] = initial_spi$

```

1: procedure CALIBRATEGAIN
2:    $iters \leftarrow 8$ 
3:    $step\_size \leftarrow 90$ 
4:   for  $i = 0, iters$  do
5:     if  $step\_size < 1$  then
6:        $step\_size \leftarrow 1$ 
7:     end if
8:     ADJUSTGAIN( $step\_size$ )
9:     for  $j = 0, NUMSENSORS$  do
10:      SENDSPI( $j, spi\_vals[j]$ )
11:    end for
12:     $step\_size \leftarrow step\_size/2$ 
13:  end for
14: end procedure

```

Require: `adcBuf` contains valid ADC data

```

15: procedure ADJUSTGAIN( $step\_size$ )
16:    $t \leftarrow (max - min)/2$ 
17:    $t_L \leftarrow (7 * t)/10$ 
18:    $t_H \leftarrow (13 * t)/10$ 
19:   for  $i = 0, NUMSENSORS$  do
20:      $v \leftarrow \text{VARIANCE}(adcBuf[i])$ 
21:      $e_v \leftarrow (t - v)/(t + v)$ 
22:      $adjustment \leftarrow step\_size * error$ 
23:     if  $v > t_L$  and  $v < t_H$  then
24:        $adjustment \leftarrow 0$ 
25:     end if
26:      $cspi \leftarrow spi\_vals[i] + adjustment$ 
27:     if  $cspi < 0$  then
28:        $cspi \leftarrow 0$ 
29:     else if  $cspi > 180$  then
30:        $cspi \leftarrow 180$ 
31:     end if
32:      $spi\_vals[i] \leftarrow cspi$ 
33:   end for
34: end procedure

```

▷ `adcBuf[i]` refers to the i^{th} boards data

Gain Control Command Block Diagram

The below block diagram outlines the logic used to execute the auto calibration command. This command was simplified using the `ADC_manager` class, however, this has caused some control inconsistencies in the ADC sample gather phase of the command.

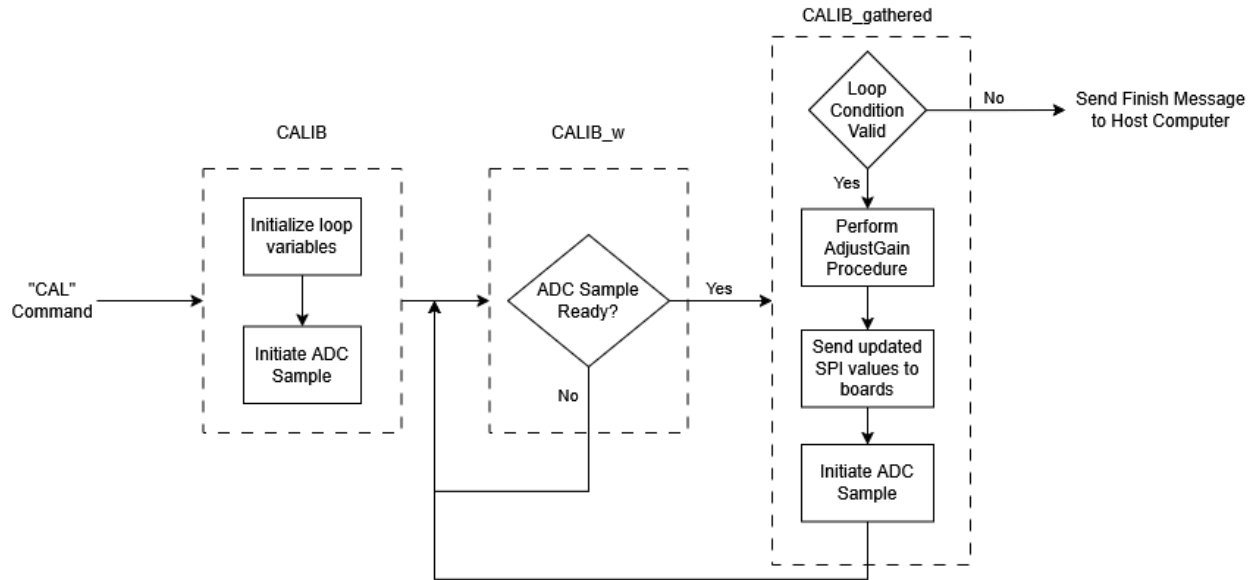


Figure A3: Calibration command logic

Firmware Next Steps

While the firmware works, incorporating more C++ features could result in a more readable codebase. By developing a consistent interface/class or namespace to manage global variables would prevent blocks of floating variables around the code that make parsing some of the functionality difficult.

While not an optimal class currently, treating each component of the firmware (state transitions, state executions, and command parsing) like the ADC_manager class could make the code more extendable in the future. If the state transition logic is left in the main while loop, breaking each command into its own class (similar to the gain_control class, but with an improved interface) could also be a worthwhile change.

Lastly, the firmware should be tested with two pingers simultaneously. This would require minor modifications to the Python utility scripts, and changing one of the parameters in the code. If the pingers are not synchronized, slight additional processing would need to be done by the host in order to remove one of the invalid results (only one peak would be found if the pingers were sending pulses at different times).

Far Field Assumption In Testing Environment

$$\text{Far Field} > 2 D^2 / \lambda$$

$$D = d(N - 1)$$

where N is the number of sensors; d is the sensor spacing

$$N=4, d=19.8 \text{ [mm]}, \lambda = c / f = 1488 \text{ [m/s]} / 30 \text{ [kHz]}$$

1	$l = \frac{1488}{30 \cdot 10^3}$	= 0.0496
2	$D = 19.8 \cdot 10^{-3}(4 - 1)$	= 0.0594
3	$\frac{2D^2}{l}$	= 0.142272580645
4	$\frac{2D^2}{l} (100) \left(\frac{1}{2.54} \right)$	= 5.60128270257

Thus, the pinger needs to be more than 5.6” away from the array to satisfy the MUSIC algorithm’s assumptions.

For each environment, we placed the array and pinger to minimize potential multipath interference. To account for echoes, the distance from the array to the “mounting wall” was considered for both environments and should satisfy:

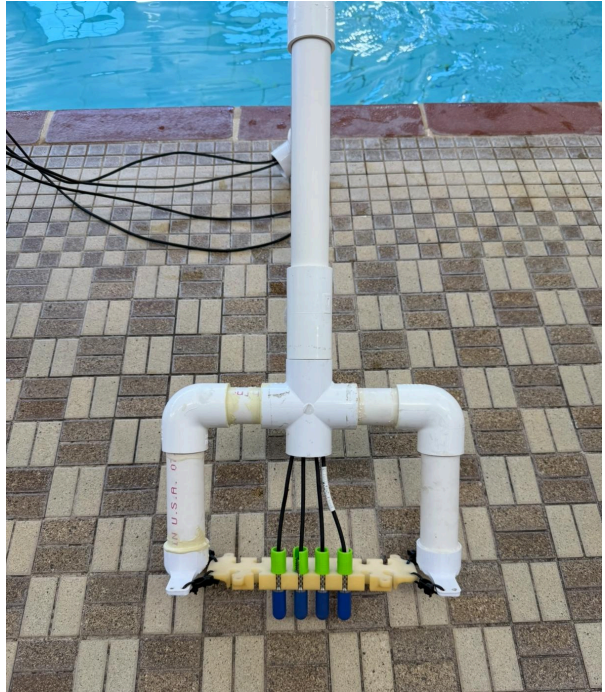
$$2 \times (\text{distance to wall}) / (\text{speed of wave}) > (\# \text{ of samples}) \times (1 / \text{sampling frequency})$$

For our PVC array and mount, we were always at least 10” from the wall (in the pool and the tank). Below is the calculation showing the first 128 samples of the rising edge satisfy this requirement:

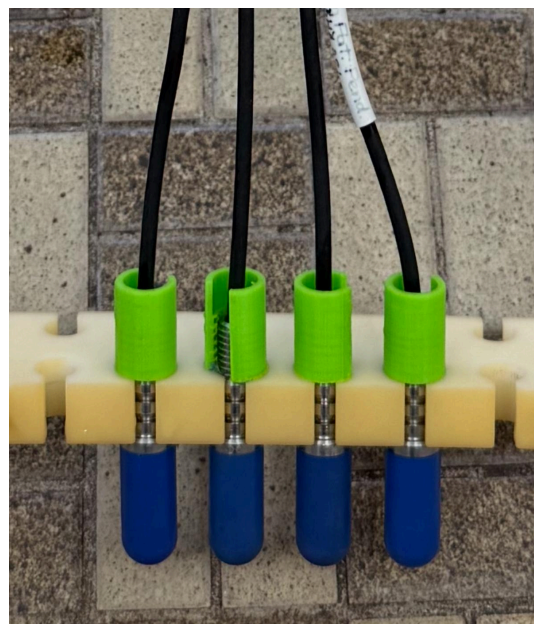
5	$2 \left(1488 \left(\frac{100}{2.54} \right) \right)^{-1} (10)$	= 0.000341397849462
6	$128 \cdot \frac{1}{400 \cdot 10^3}$	= 0.00032

Test Equipment Used

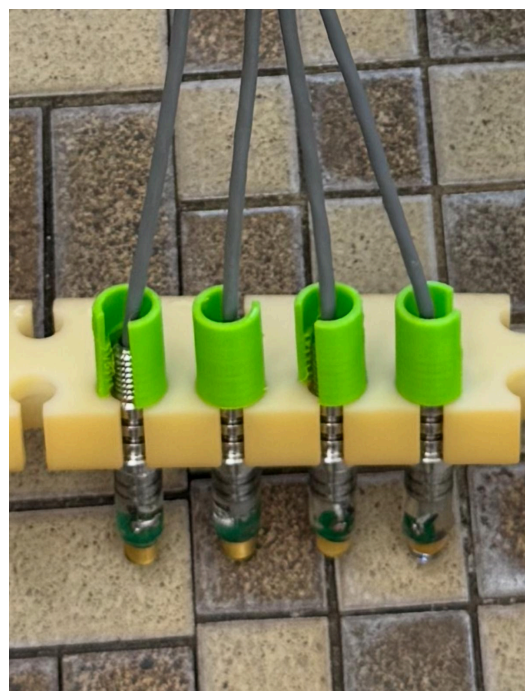
Table A3 outlines the hardware utilized during system testing.

Table A3 : Equipment Used While Testing	
Equipment	Description
PVC mount + Array	 19.8 mm spacing; 4 sensors; kept receivers mounted approximately 10" from the wall

Receivers (sensors)

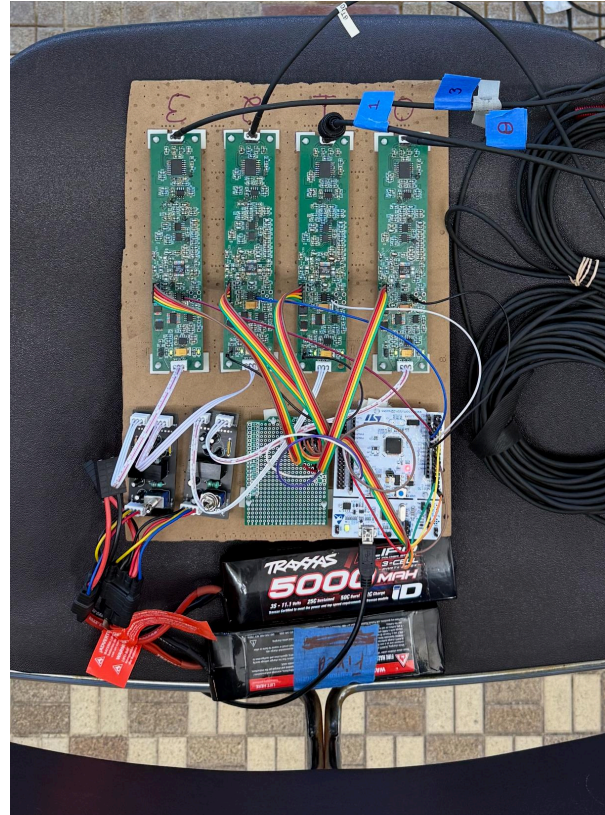


BTech 1201



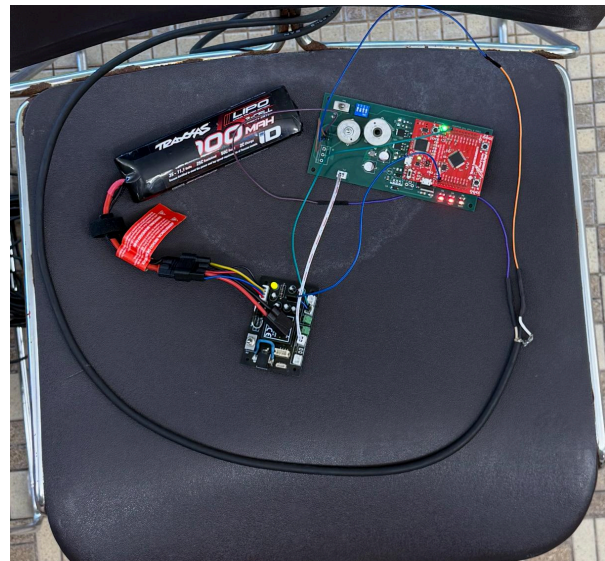
Lehigh 200 kHz

RX hardware board



STM32, LNA boards (x4), LiPo (x2)

Acoustic Pinger



Transducer (black cable), Driver board, LiPo

Host Computer



Laptop running Ubuntu 24.04

Oscilloscope [Tank test]



Troubleshoot hardware; verify results